COMPUTER NETWORKS · SEMESTER 1 · LECTURE 3

The application layer: sockets, the Web and HTTP/1.1

Week 3 teaching pack: lecture notes, seminar, Linux laboratory and Windows laboratory (two hours each)

Assoc. Prof. Nicolaie Popescu-Bodorin, PhD, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Part	Contents	Duration
A	Lecture 3 — The application layer: sockets, the Web and HTTP/1.1	2 h
B	Seminar 3 — HTTP timing, messages and byte streams	2 h
C	Laboratory 3 (Linux) — Sockets I: a TCP client and server in C	2 h
D	Laboratory 3 (Windows) — Sockets I: a TCP client and server with Winsock	2 h

Part A · Lecture 3

After this lecture you should be able to

- compare client–server and peer-to-peer application architectures;
- explain how a process is addressed on the Internet, and what a socket is;
- state what an application may need from transport, and choose between TCP and UDP;
- describe the Web's building blocks: objects, URLs and the HTTP request–response exchange;
- compute page-load times for non-persistent and persistent HTTP connections;
- read and write HTTP/1.1 messages, and say how a receiver knows where a message ends;
- list the system calls a TCP client and a TCP server make, and in which order.

Plan of the two hours

Part	Topic	Time
Hour 1		
1	From the bottom of the stack to the top	5 min
2	Application architectures	10 min
3	Processes, addresses and sockets	15 min
4	What applications need from transport	15 min
5	What an application-layer protocol defines	5 min
	Break	10 min
Hour 2		
6	The Web: objects and URLs	8 min

Part	Topic	Time
7	HTTP connections and their cost in round trips	15 min
8	HTTP/1.1 messages, methods and status codes	17 min
9	The socket API, step by step; summary	10 min

1. From the bottom of the stack to the top

In the first two weeks we looked at the Internet as a whole: its parts, its layers and its performance. From today we go down the stack one layer at a time, starting from the top — the **application layer**, the layer whose protocols you use every day and the one most of you will program for the rest of your careers. Two things happen this week. In the lecture we study how network applications are organised and the protocol of the Web, HTTP. In the laboratory you write your first network programs, a TCP client and a TCP server, using the same interface every browser and web server uses.

2. Application architectures

A network application is a set of programs running on different end systems and communicating over the network. Its designer first chooses an **architecture** — how the roles are divided between the programs. Routers and switches are not involved: they do not run application code, which is why new applications can be deployed without changing the network.

	Client–server	Peer-to-peer (P2P)
Roles	An always-on **server** with a fixed, known address serves many **clients**. Clients do not talk to each other.	Intermittently connected hosts (**peers**) talk directly to each other; each is both client and server.
Examples	Web, e-mail, online banking, most cloud services	BitTorrent, parts of some video-call and software-update systems
Scaling	The service provider must buy capacity for all clients; large services use **data centres** with many servers behind one name.	**Self-scaling**: every new peer brings demand *and* capacity.
Difficulties	Cost of servers and bandwidth; a single point of control and failure.	Peers come and go, sit behind NAT, have asymmetric access links; security and incentives are hard.

Many real systems are **hybrids**: a messaging service uses servers to find users and relay offline messages, while a voice or video call may flow directly between the two phones.

3. Processes, addresses and sockets

Hosts do not communicate; **processes** do — programs running on hosts. Two processes on the same host communicate through the operating system (pipes, shared memory). Processes on different hosts communicate by exchanging **messages** across the network. In each exchange, the process that starts it is the **client** and the one waiting to be contacted is the **server**. (In a P2P file transfer, the same peer is client in one exchange and server in another.)

3.1 The socket: a door between the application and transport

A process sends and receives messages through a software interface called a **socket**. A good picture is a door: the process pushes a message out of its door and relies on the transport infrastructure on the other side to deliver it to the door of the destination process. The application developer controls everything on the application side of the socket; on the transport side, the only choices are the transport protocol and a few parameters (buffer sizes, timeouts). Section 9 and the laboratory show the socket calls in detail.

3.2 Addressing a process

To send a message to a process on another host, a sender needs two things:

- the **IP address** of the host (32 bits in IPv4, 128 bits in IPv6), which identifies the host's network interface; and
- a **port number** (16 bits), which identifies the receiving process — strictly, the receiving socket — on that host.

Well-known services use well-known ports: HTTP 80, HTTPS 443, DNS 53, SMTP 25, SSH 22. A client's operating system gives its socket a temporary **ephemeral port**. A TCP connection is identified by four numbers together, the **4-tuple** *(source IP, source port, destination IP, destination port)*. This is how a web server with thousands of open connections to port 443 tells them apart: no two have the same 4-tuple.

4. What applications need from transport

When you create a socket you choose a transport protocol. The right choice depends on what the application needs, along four dimensions:

- **Reliable data transfer.** Must every byte arrive, intact and in order? A file transfer or a bank transaction tolerates no loss. A video call tolerates some: a lost fragment of a frame is better than a late one.
- **Throughput.** Does the application need a minimum rate (a video stream at 5 Mbit/s), or can it use whatever is available (e-mail, file transfer — *elastic* applications)?
- **Timing.** Must data arrive within some delay? Interactive games and voice need tens of milliseconds; a file download does not care.
- **Security.** Confidentiality, integrity and authentication of the endpoints.

Application	Data loss	Throughput	Time-sensitive
File transfer, download	no loss	elastic	no
E-mail	no loss	elastic	no
Web pages	no loss	elastic	somewhat (seconds)
Voice and video calls	loss-tolerant	audio: kbit/s; video: Mbit/s	yes, ~100 ms
Streaming stored video	loss-tolerant	Mbit/s	yes, a few seconds
Interactive games	loss-tolerant	kbit/s and up	yes, tens of ms
Text messaging	no loss	elastic	yes and no

4.1 What the Internet offers

	TCP	UDP
Connection	**connection-oriented**: a handshake before data flows	connectionless: each datagram stands alone
Reliability	**reliable, in-order byte stream**	none: datagrams may be lost, duplicated, reordered
Flow control	yes: never overruns the receiver	no
Congestion control	yes: slows down when the network is congested	no
Message boundaries	**none** — a stream of bytes	kept: one send = one datagram
Timing, throughput guarantees	none	none
Typical users	HTTP/1.1 and HTTP/2, SMTP, SSH, file transfer	DNS, voice and video, games; QUIC (and so HTTP/3)

Neither protocol guarantees timing or throughput: the Internet is best effort, as we saw in Lecture 1. Applications that need them cope through design — buffering, adapting their rate, choosing nearby servers. Neither protocol encrypts. Security is added above transport by **TLS** (Transport Layer Security), a library that the application calls instead of the plain socket, giving an encrypted, authenticated TCP connection; HTTPS is HTTP over TLS. QUIC, the transport under HTTP/3, runs over UDP and builds reliability, congestion control and TLS encryption into one protocol.

> **Why would anyone choose UDP?**
> Because sometimes the services of TCP get in the way. TCP's retransmissions and ordering can hold back fresh data behind old lost data; its handshake costs a round trip before the first byte; its congestion control may slow a sender that would rather lose some data than arrive late. An application on UDP can add exactly the reliability it needs — or build a whole new transport, as QUIC does.

5. What an application-layer protocol defines

An application-layer protocol fixes, for the processes of one application:

- the **types of messages** exchanged — requests and responses, for example;
- the **syntax** of each type — its fields and how they are delimited;
- the **semantics** of the fields — what the information in them means;
- the **rules** for when and how a process sends messages and responds to them.

Some protocols are **open**, specified in public RFCs so that anyone can write a compatible program: HTTP, SMTP, DNS. Others are **proprietary**, such as many messaging and video-call protocols. Note the distinction between an *application* and its *application-layer protocol*: the Web is an application made of HTML documents, browsers, web servers and standards; HTTP is only the protocol by which browsers and servers exchange messages.

6. The Web: objects and URLs

A web page consists of **objects**: files such as an HTML document, images, style sheets, scripts and videos, each addressable by a **URL** (Uniform Resource Locator). Most pages are a **base HTML file** that references many other objects; a typical news page today references well over a hundred.

```
https://www.example.org:8443/courses/networks/week3.html?lang=en#exercises
\___/   \_____________/ \__/\_____________________________/ \_____/ \_______/
scheme       host        port            path               query   fragment
```

The **scheme** names the protocol (http, https). The **host** is resolved by DNS to an IP address. The **port** is optional and defaults to 80 for http and 443 for https. The **path** names the object on the server, the **query** passes parameters, and the **fragment** is used only by the browser (to scroll to a part of the page) — it is never sent to the server.

HTTP (HyperText Transfer Protocol) is the Web's application-layer protocol. The client (browser) sends a **request** for an object; the server returns a **response** containing it. HTTP/1.1 and HTTP/2 run over TCP; HTTP/3 runs over QUIC. HTTP is **stateless**: the server keeps no memory of earlier requests from the same client. Statelessness makes servers simpler and more robust; when an application needs state — a login, a shopping basket — it is added on top, with **cookies** (next week).

7. HTTP connections and their cost in round trips

How many TCP connections should a browser use to fetch a page and its objects? There are two basic answers, and the difference is measured in **round-trip times (RTT)** — the time for a small packet to go from client to server and back.

7.1 Non-persistent connections

One TCP connection per object: open the connection, send the request, receive the response, close. Each object costs **one RTT to set up TCP** (the client sends SYN and waits for SYN-ACK) plus **one RTT for the request and the start of the response**, plus the time to transmit the object:

$$\text{time per object} = 2\ \text{RTT} + \text{transmission time}$$

HTTP/1.0 worked this way. Besides the extra round trips, every connection costs the server memory and the start of every connection runs slowly while TCP probes the available rate (we see why in the transport lectures).

7.2 Persistent connections

The server leaves the TCP connection open after responding, and later requests between the same client and server reuse it. After the first object, each further object costs **one RTT** plus its transmission time. Persistent connections are the default in HTTP/1.1. The connection is closed when either side sends `Connection: close` or after a period of inactivity.

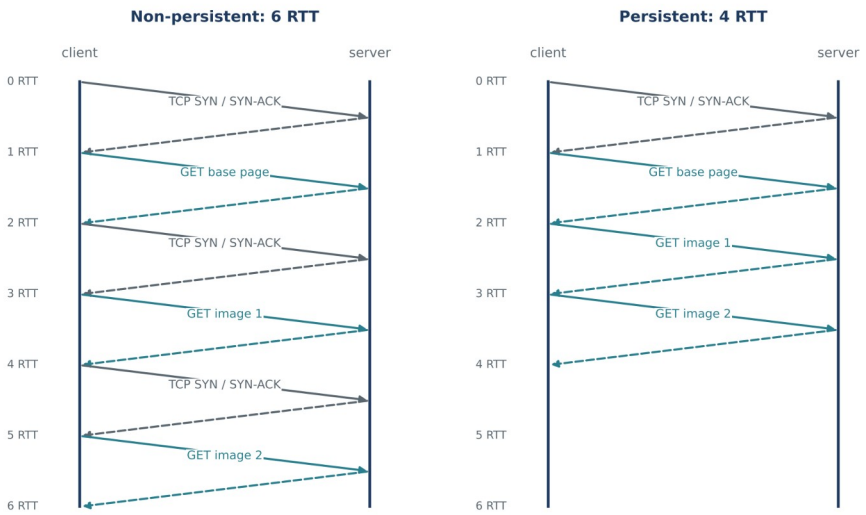


Figure 1. Fetching a base page and two images. Non-persistent: 2 RTT per object. Persistent: one handshake, then 1 RTT per object. Transmission times are ignored.

7.3 Doing several things at once

- **Parallel connections.** Browsers open several TCP connections to the same server at once (typically up to six for HTTP/1.1), fetching several objects simultaneously. This hides latency but multiplies server load and competes unfairly with other users' single connections.
- **Pipelining.** HTTP/1.1 allows a client to send several requests on one connection without waiting for the responses. Responses must come back in order, so one slow response blocks those behind it — **head-of-line blocking**. Browsers disabled pipelining in practice.
- **Multiplexing.** HTTP/2 splits requests and responses into frames and interleaves them on one TCP connection, removing head-of-line blocking at the HTTP level. HTTP/3 removes it also at the transport level, by running over QUIC. Both come next week.

> **Worked example**
>
> A page has a base HTML file and 10 small images on the same server; RTT = 50 ms; ignore transmission and DNS.
>
> **Non-persistent, one at a time:** 11 objects × 2 RTT = 22 RTT = **1100 ms**.
>
> **Non-persistent, 5 parallel connections for the images:** base 2 RTT, then two rounds of images × 2 RTT = 6 RTT = **300 ms**.
>
> **Persistent, one request at a time:** 2 RTT for the base page (including the handshake) + 10 × 1 RTT = 12 RTT = **600 ms**.
>
> **Persistent with pipelining (or HTTP/2):** 2 RTT for the base page + 1 RTT for all images = 3 RTT = **150 ms**.
>
> The round trip, not the link rate, dominates the loading of pages made of many small objects.

8. HTTP/1.1 messages

HTTP/1.1 messages are **text**: lines of ASCII characters, each ending in carriage return and line feed (CR LF, written `\r\n`). That is why you could type a request by hand in Laboratory 1. The current specifications are RFC 9110 (HTTP semantics, common to all versions) and RFC 9112 (the HTTP/1.1 message format), both from 2022.

8.1 Requests

```
GET /courses/networks/week3.html HTTP/1.1     <- request line: method, target, version
Host: www.example.org                          <- header lines: name: value
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: text/html
Accept-Language: ro, en;q=0.8
Connection: keep-alive
                                               <- empty line: end of the header
```

The **request line** gives the method, the target (the path and query of the URL) and the version. **Header lines** follow, one per line. The **Host** header is mandatory in HTTP/1.1: one server, with one IP address, may host thousands of sites, and the Host header tells it which one is meant. An **empty line** ends the header. Some requests then carry a **body** — the data of a submitted form, for example.

Method	Meaning	Safe	Idempotent
GET	Retrieve the object named by the target	yes	yes
HEAD	Like GET, but return only the header (no body)	yes	yes
POST	Submit data to be processed (a form, an order, an upload)	no	no
PUT	Create or replace the object at the target with the body	no	yes
DELETE	Delete the object at the target	no	yes
PATCH	Modify part of an object	no	no (in general)
OPTIONS	Ask which methods the server supports for the target	yes	yes

A method is **safe** if it does not change anything on the server, and **idempotent** if repeating it has the same effect as doing it once. These properties matter for the question raised in Lecture 1: a client that gets no response may repeat an idempotent request safely, but repeating a POST may place an order twice. That is why browsers ask “Confirm form resubmission?” when you reload a page obtained by POST.

8.2 Responses

```
HTTP/1.1 200 OK                                <- status line: version, code, reason
Date: Mon, 19 Oct 2026 08:00:00 GMT
Server: nginx
Last-Modified: Fri, 16 Oct 2026 14:12:30 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 5120

<!doctype html> ... (5120 bytes of body)
```

Class	Meaning	Common codes
1xx	informational	100 Continue · 101 Switching Protocols
2xx	success	**200 OK** · 201 Created · 204 No Content
3xx	redirection: look elsewhere	**301 Moved Permanently** · 302 Found · **304 Not Modified**
4xx	client error	400 Bad Request · 401 Unauthorized · 403 Forbidden · **404 Not Found** · 405 Method Not Allowed
5xx	server error	500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable

8.3 Where does a message end?

TCP delivers a **stream of bytes** with no boundaries between messages (section 9). A receiver therefore needs a rule to know where each HTTP message ends — otherwise, on a persistent connection, it cannot tell the end of one response from the start of the next. HTTP/1.1 uses three rules:

1. **The header ends at the first empty line.** The receiver reads lines until it finds `\r\n\r\n`.
2. **A body of known length** is announced with **Content-Length**: the receiver reads exactly that many bytes after the header.
3. **A body of unknown length** — generated while it is being sent — uses **Transfer-Encoding: chunked**: the body is sent as chunks, each preceded by its length in hexadecimal on a line of its own, and ended by a chunk of length 0. (A server may also simply close the connection at the end of the body, but then the connection cannot be reused.)

```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked

7
Network
9
s, week 3
0

```

The body above is the 16 bytes “Networks, week 3”. Every protocol built on TCP must solve the same **framing** problem. You will meet it in the laboratory today, and solve it in three ways: fixed-length messages, lines ending in `\n`, and a length prefix.

9. The socket API, step by step

The **Berkeley sockets** interface appeared in 4.2BSD UNIX in 1983 and is today the interface of every operating system; Windows implements it as **Winsock**, with small differences that the Windows laboratory sheet lists. Figure 2 shows the calls made by a TCP server and a TCP client.

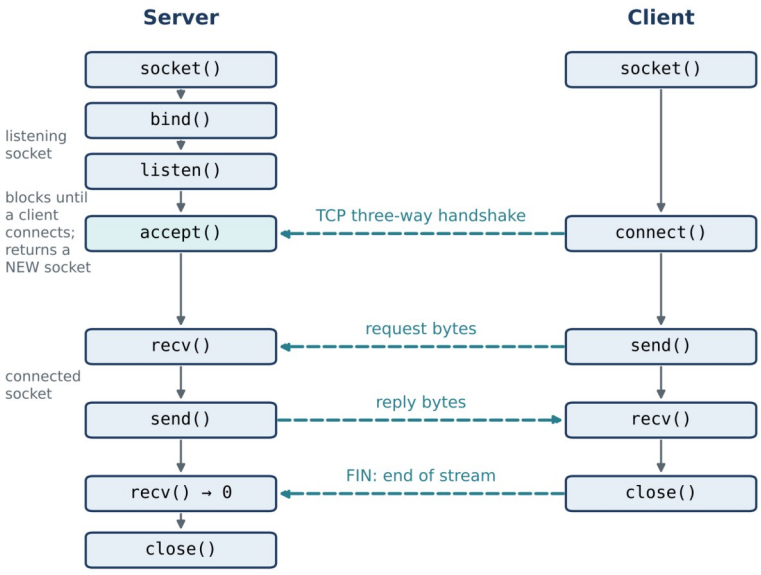


Figure 2. The system calls of a TCP server and client. accept() returns a new, connected socket for each client; the listening socket keeps waiting for others.

Call	Who	What it does
`socket()`	both	Creates a socket: address family (IPv4/IPv6) and type (stream = TCP, datagram = UDP). Returns a descriptor.
`bind()`	server	Attaches the socket to a local address and port, such as 0.0.0.0:8080.
`listen()`	server	Marks the socket as passive: it will accept connections. Sets the length of the queue of pending connections.
`connect()`	client	Starts the TCP handshake with the server's address and port. The OS picks an ephemeral local port.
`accept()`	server	Waits for a completed connection and returns a **new socket** for it; the listening socket stays open.
`send()` / `recv()`	both	Write bytes into, and read bytes from, the connection. `recv()` returns 0 when the peer has closed.
`close()`	both	Releases the socket; for TCP, sends FIN to end the stream in this direction.

9.1 Three facts that surprise beginners

- **A stream has no message boundaries.** Two `send()` calls of 5 bytes may arrive as one `recv()` of 10 bytes, or as `recv()` calls of 3 and 7. One `send()` of 300 000 bytes may arrive in five pieces of different sizes. A program must loop until it has the bytes it expects, and the protocol must define where a message ends.
- **`send()` may accept fewer bytes than asked.** It returns the number of bytes copied into the operating system's buffer; the rest must be sent again. A correct program wraps it in a loop (`send_all` in the laboratory; `sendall` in Python).
- **Byte order matters.** Port numbers, addresses and any binary integers sent over the network use **network byte order** (big-endian). The functions `htons`, `htonl`, `ntohs`, `ntohl` convert between host and network order; forgetting them is the classic bug that makes a server listen on port 36895 instead of 8080 on an x86 machine.

Summary

- Network applications run only on end systems, organised as **client–server**, **peer-to-peer** or a hybrid.
- Processes communicate through **sockets**; a process is addressed by **IP address + port**; a TCP connection is identified by its **4-tuple**.
- Applications differ in their needs for **reliability, throughput, timing and security**. **TCP** gives a reliable, ordered byte stream with flow and congestion control; **UDP** gives bare datagrams; **TLS** adds security; neither gives timing guarantees.
- An application-layer protocol defines **message types, syntax, semantics and rules**.
- The Web is made of **objects** named by **URLs** and fetched with **HTTP**, a **stateless** request–response protocol.
- **Non-persistent** HTTP costs 2 RTT per object; **persistent** HTTP costs 1 RTT per object after the first. Parallel connections, pipelining and multiplexing reduce page load times further.
- HTTP/1.1 messages are text: request or status line, headers, empty line, optional body. The body's end is given by **Content-Length** or **chunked** encoding — the **framing** problem every TCP protocol must solve.

- TCP servers call `socket`, `bind`, `listen`, `accept`; clients `socket`, `connect`; both `send`, `recv`, `close`.

Key terms

client–server · peer-to-peer · self-scaling · process · socket · IP address · port number · ephemeral port · 4-tuple · reliable data transfer · elastic application · throughput · timing · TCP · UDP · TLS · QUIC · application-layer protocol · open / proprietary protocol · object · URL · scheme · host · path · query · fragment · HTTP · stateless · non-persistent / persistent connection · RTT · parallel connections · pipelining · head-of-line blocking · multiplexing · request line · header line · Host header · method · safe · idempotent · status line · status code · Content-Length · chunked transfer encoding · framing · Berkeley sockets · Winsock · listening socket · connected socket · network byte order

Review questions

1. Why can a new network application be deployed without any change to the Internet's routers?
2. In a BitTorrent download, which peer is the client and which the server? Can the answer change during the download?
3. A web server has 10 000 open TCP connections, all to its port 443. How does it deliver an arriving segment to the right connection?
4. Give one application for which UDP is a better choice than TCP, and explain which TCP service would get in its way.
5. Neither TCP nor UDP encrypts. How does HTTPS protect a connection, and at which layer?
6. Which part of a URL is never sent to the server? Why?
7. Why is the Host header mandatory in HTTP/1.1?
8. Explain why a browser may safely repeat a GET request but should warn before repeating a POST.
9. Give two ways in which an HTTP/1.1 receiver can know where a response body ends.
10. What does `accept()` return, and why is it not the listening socket itself?

Exercises

1. A page consists of a base HTML file and 4 images, all on one server. RTT = 80 ms; each object takes 10 ms to transmit; ignore DNS. Compute the page-load time with (a) non-persistent connections, one at a time; (b) persistent connections without pipelining; (c) persistent connections with pipelining.
2. Repeat exercise 1(a) when the browser opens up to 4 parallel non-persistent connections for the images.
3. Decode this chunked body and give its length: `5\r\nHello\r\n8\r\n, world!\r\n0\r\n\r\n`.
4. A client sends a 1 000-byte request with a single `send()` call. Is the server guaranteed to receive it with a single `recv()`? What if the request is 10 bytes? Explain.
5. On a little-endian machine, a programmer writes `addr.sin_port = 8080;` instead of `htons(8080)`. 8080 is 0x1F90. On which port does the server actually listen?

Answers

1. (a) 5 objects × (2 × 80 + 10) = **850 ms**. (b) Base page: 2 × 80 + 10 = 170 ms; 4 images × (80 + 10) = 360 ms; total **530 ms**. (c) Base page 170 ms; the 4 requests go out together, the first response arrives after 80 ms and the four are transmitted back to back: 80 + 4 × 10 = 120 ms; total **290 ms**.
2. Base page 170 ms, then all 4 images in parallel, each 2 × 80 + 10 = 170 ms (assuming the transmissions do not slow each other): **340 ms**.
3. “Hello” (5 bytes) followed by “, world!” (8 bytes): the body is “Hello, world!”, **13 bytes**.
4. No, in both cases. TCP is a byte stream; the receiver may get the bytes in several pieces, depending on segment sizes, timing and buffer sizes. Small messages *usually* arrive whole, which is why the bug of assuming so often goes unnoticed in testing and appears in production.
5. The bytes 0x90 0x1F are interpreted in network order as **0x901F = 36 895**.

Reading and next steps

Required

- Kurose and Ross, *Computer Networking: A Top-Down Approach*, 8th edition, Chapter 2: sections on principles of network applications and on the Web and HTTP; the section on socket programming.

Recommended

- B. Hall, *Beej’s Guide to Network Programming* (free online), chapters 2–6.
- W. R. Stevens, B. Fenner, A. M. Rudoff, *UNIX Network Programming, Volume 1*, 3rd edition, chapters 1, 4 and 5.
- RFC 9110, “HTTP Semantics”, sections 9 (methods) and 15 (status codes); RFC 9112, “HTTP/1.1”, sections 2–3 and 6–7 (message format and framing).

Laboratory 3 (this week)

Sockets I: you build, run and break a TCP echo client and server in C, observe how a byte stream is cut into pieces, and implement two framing methods.

Next lecture

The Web, continued: cookies and state, web caching and conditional GET, HTTP/2 and HTTP/3, and content distribution networks.

PART B · WEEK 3

Seminar 3: HTTP timing, messages and byte streams

Two-hour problem session (2 × 50 minutes) accompanying Lecture 3

The seminar turns Lecture 3 into calculations and careful reading of messages. Work in pairs; one pair presents each problem.

By the end of the seminar you should be able to

- choose a transport service from an application's requirements;
- compute page-load times under different HTTP connection strategies;
- read HTTP/1.1 requests and responses, and decode chunked bodies;
- match situations with HTTP status codes and judge whether a request may be repeated;
- list the sockets and 4-tuples on a busy server;
- design a framing rule for a protocol built on a byte stream.

Plan

Step	Activity	Time
A	Problem 1: which transport?	10 min
B	Problems 2–3: how long does a page take?	25 min
C	Problem 4: reading an HTTP exchange	15 min
	Break	10 min
D	Problems 5–6: framing and chunked bodies	15 min
E	Problems 7–8: status codes and repeatable requests	10 min
F	Problems 9–10: sockets, 4-tuples and a framing design	15 min
	Homework	(in F)

A. Which transport?

Problem 1

For each application, state its needs (loss tolerance, throughput, timing) and choose TCP, UDP or "UDP with something built on top". Justify in one sentence.

	Application
a	Remote login to a server (SSH)
b	A multiplayer racing game sending each car's position 30 times per second
c	Uploading a 2 GB backup to a cloud service
d	A video call between two phones
e	Asking a DNS server for the address of a name

	Application
f	A sensor reporting the temperature every minute over a lossy radio link

Solution

(a) No loss, little throughput, interactive but tolerant of tens of ms: **TCP**. (b) Loss-tolerant — an old position is useless once a newer one exists — and time-sensitive: **UDP**; TCP would delay new positions behind retransmitted old ones. (c) No loss, elastic: **TCP**. (d) Loss-tolerant, time-sensitive, needs Mbit/s: **UDP** with a real-time protocol on top (RTP, as in WebRTC). (e) One small request and reply; the client retries itself if no answer comes: **UDP** (TCP for large answers). (f) A single reading per minute, losing one is acceptable: **UDP**; if every reading matters, add acknowledgements and retries on top, or use TCP.

B. How long does a page take?

Problem 2

A page has a base HTML file of 50 KB and 6 images of 100 KB each, all on one server. The client's access link is 20 Mbit/s and is the bottleneck; RTT = 40 ms. Ignore DNS and header sizes; 1 KB = 1000 bytes.

1. What is the transmission time of the base file? Of one image?
2. Compute the page-load time with non-persistent connections, one at a time.
3. With persistent connections, without pipelining.
4. With persistent connections and pipelining (all image requests sent at once).
5. Which matters more here: the RTT or the link rate? How would your answer change for a page of 60 images of 2 KB each?

Solution

1. Base: 400 000 bits / 2×10^7 bit/s = 20 ms. Image: 800 000 / 2×10^7 = 40 ms.

2. Base: 2 × 40 + 20 = 100 ms. Each image: 2 × 40 + 40 = 120 ms; 6 images 720 ms. Total **820 ms**.

3. Base 100 ms; each image 40 + 40 = 80 ms; total 100 + 480 = **580 ms**.

4. Base 100 ms; then 40 ms for the requests to arrive and the first bits to return, plus 6 × 40 ms transmission: 280 ms. Total **380 ms**.

5. Here transmission (240 ms for the images) and RTTs (40 ms each) are comparable. With 60 small images, each takes 0.8 ms to transmit: non-persistent costs 60 × (80 + 0.8) ≈ 4.8 s, persistent without pipelining 60 × 40.8 ≈ 2.4 s, pipelined only about 40 + 48 = 88 ms after the base page. With many small objects the **round trips** dominate.

Problem 3

A browser loads the page of Problem 2 with up to **6 parallel non-persistent connections** for the images. The six transfers share the 20 Mbit/s access link equally.

1. How long does the page take?
2. Why is the answer not 6 times better than in Problem 2.2?

> **Solution**
>
> 1. Base 100 ms. Then six connections at once: 1 RTT handshake (40 ms) + 1 RTT for request and first bits (40 ms) + transmitting 6 × 100 KB = 4.8 Mbit over 20 Mbit/s = 240 ms. Total 100 + 80 + 240 = **420 ms**.
> 2. Parallelism hides round trips, not transmission: the six images still share one 20 Mbit/s link, so their 240 ms of transmission does not shrink.

C. Reading an HTTP exchange

Problem 4

A capture shows this request and response on one TCP connection.

```
GET /files/notes.pdf HTTP/1.1
Host: cn.example.edu
User-Agent: curl/8.5.0
If-Modified-Since: Fri, 16 Oct 2026 14:12:30 GMT
Accept: */*

HTTP/1.1 304 Not Modified
Date: Mon, 19 Oct 2026 08:00:02 GMT
Last-Modified: Fri, 16 Oct 2026 14:12:30 GMT
ETag: "5f3a-1e2b4"
```

1. What is the full URL of the requested object?
2. Which program sent the request? Is the connection persistent? How do you know?
3. What does the header If-Modified-Since ask, and what does the server answer?
4. How many bytes of body does the response carry? How does the client know?
5. Why might a client send such a request at all?

> **Solution**
>
> 1. http://cn.example.edu/files/notes.pdf — the scheme is http because this is plain HTTP on a TCP connection (an HTTPS exchange would be encrypted).
> 2. curl, version 8.5.0 (User-Agent). Persistent: HTTP/1.1 is persistent by default and neither side sent `Connection: close`.
> 3. “Send the object only if it changed after 16 October, 14:12:30.” The server answers **304 Not Modified**: the client’s copy is still valid.
> 4. None: a 304 response never has a body; the header ends at the empty line.
> 5. The client has a cached copy and wants to check that it is still fresh without downloading it again. This **conditional GET** is the subject of next week’s lecture on caching.

D. Framing

Problem 5 — chunked bodies

Decode the following body (`\r\n` marks CR LF). What is its length, and what does the last line mean?

```
1a\r\n
abcdefghijklmnopqrstuvwxyz\r\n
a\r\n
0123456789\r\n
0\r\n
\r\n
```

> **Solution**
> Chunk sizes are hexadecimal: 0x1a = 26 bytes (the alphabet), then 0xa = 10 bytes (the digits). Total **36 bytes**. The chunk of size 0 followed by an empty line ends the body; the connection can now carry the next response.

Problem 6 — what the receiver sees

A client calls `send(fd, "HELLO", 5)` and immediately `send(fd, "WORLD", 5)` on a TCP socket. The server calls `recv(fd, buf, 100)` repeatedly.

1. List three different sequences of return values the server could observe.
2. The protocol designer wanted two separate messages. Propose two ways to make the boundary visible to the server, and give the exact bytes sent in each case.

> **Solution**
> 1. For example: 10 ("HELLOWORLD" in one call); 5 and 5; 3 and 7 ("HEL", "LOWORLD"). Any split of the 10 bytes, in order, is possible — but never a reordering or a loss.
> 2. **Delimiter:** end each message with a newline: `HELLO\nWORLD\n` (12 bytes); the server reads until each `\n`. The delimiter must never appear inside a message, or must be escaped. **Length prefix:** send a 4-byte length in network byte order before each message: `00 00 00 05 H E L L O 00 00 00 05 W O R L D` (18 bytes); the server reads 4 bytes, then exactly that many. Works for any content, including binary data.

E. Status codes and repeatable requests

Problem 7

Which status code should a server send in each situation?

1. The site moved for good from http:// to https://.
2. A client asks for /admin without having logged in.
3. A logged-in student asks for /admin, which only teachers may see.
4. The client sends `DELETE /index.html` to a server that never deletes pages.
5. A new account was created by a POST request.
6. The application behind the web server crashed while handling the request.
7. The client's request line is `GTE / HTTP/1.1`.

> **Solution**
> 1. **301 Moved Permanently** (with a Location header). 2. **401 Unauthorized** (authentication needed). 3. **403 Forbidden** (authenticated, but not allowed). 4. **405 Method Not Allowed** (or 501 Not Implemented if the server does not know the method at all). 5. **201 Created**. 6. **500 Internal Server Error** (or 502 Bad Gateway if a proxy in front reports it). 7. **400 Bad Request** — or 501, since GTE is an unknown method.

Problem 8

A mobile app sends a request and the connection drops before any response arrives. For each request, may the app repeat it automatically?

1. `GET /timetable?group=CS2`
2. `PUT /profile/photo` with a new photo

3. `DELETE /cart/items/17`
4. `POST /orders` with the contents of the shopping cart

> **Solution**
> 1. Yes: safe and idempotent. 2. Yes: idempotent — uploading the same photo twice leaves the same result. 3. Yes: idempotent — the second DELETE finds nothing to delete (it may answer 404, which the app must accept). 4. **No**: the first order may have been placed; repeating could order twice. The usual fix is an **idempotency key**: the app sends a unique identifier with the order, and the server ignores a second order with the same identifier — the request identifier of Seminar 1, Problem 6.

F. Sockets

Problem 9

A web server at 203.0.113.10 listens on port 80. Three connections are open: two from a laptop at 198.51.100.7 (two browser tabs) and one from a phone at 192.0.2.33.

1. How many sockets does the server process have open?
2. Write plausible 4-tuples for the three connections, from the server's point of view.
3. The two connections from the laptop have the same source IP and the same destination IP and port. How does the server tell them apart?

> **Solution**
> 1. Four: the listening socket and one connected socket per client, as returned by `accept()`.
> 2. For example (source IP, source port, destination IP, destination port): (198.51.100.7, 52314, 203.0.113.10, 80); (198.51.100.7, 52316, 203.0.113.10, 80); (192.0.2.33, 40122, 203.0.113.10, 80).
> 3. By the source port: the laptop's operating system gave each socket a different ephemeral port. Two connections can never share all four numbers.

Problem 10 — design

You must design the protocol of a small file-upload service over TCP. The client sends a file name and the file's bytes; the server answers OK or an error. The file may contain any bytes, including newlines and zeros. Specify the exact message format, so that two students could implement client and server independently.

> **One possible answer**
> **Request:** a header line in text, then the raw bytes: PUT `<name> <size>\r\n` followed by exactly `<size>` bytes. The name may not contain spaces or CR LF; size is decimal.
> **Response:** one line: OK`\r\n`, or ERR `<code> <text>\r\n` (for example ERR `413 too large`).
> **Framing:** the header ends at the first CR LF (a delimiter, safe because names cannot contain it); the body's end is given by its length (a length field, safe for binary data) — exactly the two rules of HTTP/1.1.
> **Rules:** several requests may follow one another on one connection; the server answers each in order; either side may close after a response. Repeating a PUT of the same file is harmless: the request is idempotent.

Homework

- Solve exercises 1–5 at the end of Lecture 3 without looking at the answers.
- With `curl -v http://example.org/` and then `curl -v --http1.0 http://example.org/`, compare the two requests and the Connection behaviour.
- Write down, for the protocol of Problem 10, what the server should do if the connection closes before `<size>` bytes have arrived.

PART C · WEEK 3

Laboratory 3 (Linux): Sockets I — a TCP client and server in C

Two-hour laboratory sheet (2 × 50 minutes) accompanying Lecture 3

Today you write the programs that until now you only observed. You will build a TCP client and server in C, watch the calls of Lecture 3 at work, see a byte stream break into pieces, and implement two ways of putting message boundaries back. All programs in this sheet compile without warnings and were tested on Ubuntu 24.04; they are also provided as files.

By the end of the laboratory you should be able to

- write, compile and run a TCP client and an iterative TCP server in C;
- resolve names with `getaddrinfo` and handle IPv4 and IPv6 in a client;
- observe listening and connected sockets and their 4-tuples with `ss`;
- explain, with measurements, why `recv` must be called in a loop;
- implement line-based and length-prefixed framing;
- recognise the common errors: connection refused, address in use, peer closed, broken pipe.

Before you start

- **Environment.** The course VM or any Linux with a C compiler. Install the tools once:

```
sudo apt install -y build-essential netcat-openbsd manpages-dev manpages-posix-dev
```

- **Files.** Unpack the archive of this laboratory into a folder `lab3/`; it contains `netio.h`, `echo_client.c`, `echo_server.c`, `upper_server.c`, `blast.c`, `msg_client.c`, `msg_server.c` and a `Makefile`. Build everything with `make`.
- **Manual pages.** `man 2 socket`, `man 2 bind`, `man 2 accept`, `man 2 recv`, `man 3 getaddrinfo`, `man 7 tcp`. Read the RETURN VALUE and ERRORS sections: they are where network programs fail.
- **Ports.** Use a port between 5000 and 60000. If a colleague on the same machine uses yours, pick another.
- **What to hand in.** Your modified sources and a short report with answers to **Q1–Q16**.

Plan

Step	Activity	Time
1	Warm-up with netcat: a server without writing code	10 min
2	Read and run the echo client	15 min
3	Read and run the echo server	20 min
4	Sockets as the operating system sees them	5 min
	Break	10 min
5	A stream is not a sequence of messages	15 min
6	Framing with lines: the upper-case server	10 min

Step	Activity	Time
7	Framing with a length prefix	15 min
8	Breaking things on purpose	10 min

Task 1 — A server without writing code

In terminal A start a listening `nc`; in terminal B connect to it; type in either window:

```
nc -l 5000                # terminal A: listen on port 5000
nc localhost 5000         # terminal B: connect
ss -tln | grep 5000       # terminal C: listening sockets
ss -tn  | grep 5000       # terminal C: connected sockets and their 4-tuples
```

- **Q1.** What does `ss -tln` show before the client connects, and what does `ss -tn` show after? Write the 4-tuple of the connection. Which port did the client get, and who chose it?
- **Q2.** Close terminal B with Ctrl+D. What happens to the server? Which TCP message told it?

Task 2 — The echo client

All programs share the helpers in `netio.h`. `send_all` and `recv_all` loop until all the requested bytes are written or read; `send_msg` and `recv_msg` are used in Task 7.

```
/* netio.h - helpers for TCP byte streams (Computer Networks, Lab 3) */
#ifndef NETIO_H
#define NETIO_H
#include <stdint.h>
#include <errno.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

/* Write exactly len bytes. Returns 0 on success, -1 on error. */
static inline int send_all(int fd, const void *buf, size_t len) {
    const char *p = buf;
    while (len > 0) {
        ssize_t n = send(fd, p, len, 0);
        if (n < 0) { if (errno == EINTR) continue; return -1; }
        p += n; len -= (size_t)n;
    }
    return 0;
}

/* Read exactly len bytes. Returns 1 on success, 0 if the peer closed first, -1 on error.
*/
static inline int recv_all(int fd, void *buf, size_t len) {
    char *p = buf;
    while (len > 0) {
        ssize_t n = recv(fd, p, len, 0);
        if (n < 0) { if (errno == EINTR) continue; return -1; }
        if (n == 0) return 0;
        p += n; len -= (size_t)n;
    }
    return 1;
}

/* Length-prefixed messages: 4-byte length in network byte order, then the payload. */
static inline int send_msg(int fd, const void *buf, uint32_t len) {
    uint32_t netlen = htonl(len);
    if (send_all(fd, &netlen, sizeof netlen) < 0) return -1;
    return send_all(fd, buf, len);
```

```
}

/* Returns the payload length, 0 if the peer closed, -1 on error or if the message exceeds
cap. */
static inline long recv_msg(int fd, void *buf, uint32_t cap) {
    uint32_t netlen;
    int r = recv_all(fd, &netlen, sizeof netlen);
    if (r <= 0) return r;
    uint32_t len = ntohl(netlen);
    if (len > cap) return -1;
    r = recv_all(fd, buf, len);
    return r <= 0 ? -1 : (long)len;
}
#endif
```

The client resolves the server's name with `getaddrinfo`, tries each address it gets (IPv6 and IPv4) until `connect` succeeds, then sends each line it reads from the keyboard and prints the echo:

```
/* echo_client.c - send each line of standard input to a TCP server, print what comes back.
   Usage: ./echo_client HOST PORT */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netdb.h>
#include <sys/socket.h>
#include "netio.h"

int main(int argc, char *argv[]) {
    if (argc != 3) { fprintf(stderr, "usage: %s HOST PORT\n", argv[0]); return 1; }

    struct addrinfo hints = {0}, *res, *ai;
    hints.ai_family = AF_UNSPEC;          /* IPv4 or IPv6 */
    hints.ai_socktype = SOCK_STREAM;      /* TCP */
    int err = getaddrinfo(argv[1], argv[2], &hints, &res);
    if (err) { fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(err)); return 1; }

    int fd = -1;
    for (ai = res; ai != NULL; ai = ai->ai_next) {          /* try each address in turn */
        fd = socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol);
        if (fd < 0) continue;
        if (connect(fd, ai->ai_addr, ai->ai_addrlen) == 0) break;
        close(fd); fd = -1;
    }
    freeaddrinfo(res);
    if (fd < 0) { perror("connect"); return 1; }

    char line[1024], reply[1024];
    while (fgets(line, sizeof line, stdin) != NULL) {
        size_t len = strlen(line);
        if (send_all(fd, line, len) < 0) { perror("send"); break; }
        /* The echo has the same length as what we sent, but may arrive in pieces. */
        int r = recv_all(fd, reply, len);
         if (r <= 0) { fprintf(stderr, r == 0 ? "server closed the connection\n" : "recv
failed\n"); break; }
        fwrite(reply, 1, len, stdout);
    }
    close(fd);
    return 0;
}
```

Test it first against `nc -l 5000` from Task 1 (type the echo by hand in the `nc` window), then go on.

- **Q3.** Why does the client call `recv_all(fd, reply, len)` rather than a single `recv`? What would go wrong with a single call?

- **Q4.** Run `./echo_client localhost 5000` while nothing listens on port 5000. What error do you get, and which TCP message caused it?

Task 3 — The echo server

```
/* echo_server.c - iterative TCP echo server: one client at a time.
   Usage: ./echo_server PORT */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include "netio.h"

int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "usage: %s PORT\n", argv[0]); return 1; }

    setvbuf(stdout, NULL, _IOLBF, 0);                        /* print each line at once */
    int lfd = socket(AF_INET, SOCK_STREAM, 0);               /* 1. create */
    if (lfd < 0) { perror("socket"); return 1; }
    int on = 1;
    setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &on, sizeof on); /* allow quick restarts */

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);                 /* all local addresses */
     addr.sin_port = htons((uint16_t)atoi(argv[1]));             /* host -> network byte
order */
    if (bind(lfd, (struct sockaddr *)&addr, sizeof addr) < 0) { perror("bind"); return 1;
} /* 2. bind */
    if (listen(lfd, 8) < 0) { perror("listen"); return 1; }  /* 3. listen, backlog 8 */
    printf("listening on port %s\n", argv[1]);

    for (;;) {
        struct sockaddr_in peer; socklen_t plen = sizeof peer;
        int cfd = accept(lfd, (struct sockaddr *)&peer, &plen); /* 4. accept: a NEW socket
*/
        if (cfd < 0) { perror("accept"); continue; }
        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &peer.sin_addr, ip, sizeof ip);
        printf("connection from %s:%u\n", ip, ntohs(peer.sin_port));

        char buf[65536]; ssize_t n; long total = 0;
        while ((n = recv(cfd, buf, sizeof buf, 0)) > 0) {      /* 5. read what has arrived */
            printf("  recv returned %zd bytes\n", n);
            total += n;
            if (send_all(cfd, buf, (size_t)n) < 0) { perror("send"); break; }
        }
        printf("client closed after %ld bytes\n", total);
       close(cfd);                                             /* 6. close this connection */
    }
}

./echo_server 5000                      # terminal A
./echo_client localhost 5000            # terminal B: type a few lines
./echo_client localhost 5000            # terminal C, while B is still connected
```

- **Q5.** Match each numbered step of the server with the calls of Figure 2 in Lecture 3. What does the server print for each client, and where does the client's port number come from?

- **Q6.** The second client (terminal C) connects but gets no echo until the first one quits. Why? Is its TCP connection established while it waits? Check with `ss -tn`. (This is the cost of an *iterative* server; Laboratory 5 builds concurrent ones.)
- **Q7.** Remove `htons` from the line that sets `sin_port`, rebuild, and run the server with port 5000. On which port does it really listen (`ss -tln`)? Explain the number.

Task 4 — Sockets as the operating system sees them

```
ss -tnp | grep -E '5000|echo'            # -p shows the owning process
ls -l /proc/$(pgrep -n echo_server)/fd   # a socket is a file descriptor
```

- **Q8.** While one client is connected, how many sockets does `echo_server` hold, and in which states? Which file descriptor numbers do they have?

Task 5 — A stream is not a sequence of messages

`blast` sends N bytes with **one** call to `send_all` and then closes its sending direction. Run it against the echo server and watch what the server prints:

```
./blast localhost 5000 300000

connection from 127.0.0.1:51304
  recv returned 65482 bytes
  recv returned 65536 bytes
  recv returned 65536 bytes
  recv returned 65536 bytes
  recv returned 37910 bytes
client closed after 300000 bytes
```

A real run on the loopback interface; your sizes will differ from run to run.

- **Q9.** Repeat three times and with N = 10, 5000 and 3 000 000. One send — how many receives? Are the sizes always the same? What limits the size of one `recv`?
- **Q10.** Change `sizeof buf` in the server to 7 and send the line `hello world` from `echo_client`. What does the server print now? Does the client still receive the full echo — and why?

Task 6 — Framing with lines

`upper_server` implements a line-based protocol: for each line ending in `\n` it replies with the line in upper case. Its loop keeps a buffer, because one `recv` may contain half a line, a whole line or several lines:

```
    for (;;) {
        int cfd = accept(lfd, NULL, NULL);
        if (cfd < 0) continue;
        char buf[4096]; size_t used = 0; ssize_t n;
        while ((n = recv(cfd, buf + used, sizeof buf - used, 0)) > 0) {
            used += (size_t)n;
            char *nl;
            /* A single recv may hold part of a line, one line, or several lines. */
            while ((nl = memchr(buf, '\n', used)) != NULL) {
                size_t linelen = (size_t)(nl - buf) + 1;
                  for (size_t i = 0; i < linelen; i++) buf[i] = (char)toupper((unsigned
char)buf[i]);
                if (send_all(cfd, buf, linelen) < 0) break;
                memmove(buf, buf + linelen, used - linelen);   /* keep the rest for later
*/
                used -= linelen;
            }
```

```
            if (used == sizeof buf) { const char *e = "ERR line too long\n"; send_all(cfd,
e, strlen(e)); break; }
        }
        close(cfd);

./upper_server 5001
printf 'abc\ndef\nghi' | nc -N localhost 5001
```

- **Q11.** Why is "ghi" never answered? What should a protocol say about an incomplete last line?
- **Q12.** What happens if a client sends a line longer than 4096 bytes? Is the server's behaviour a good protocol decision? Propose a better one.

Task 7 — Framing with a length prefix

Lines fail for binary data, which may contain `\n`. The general solution is to send the length of each message first. Study `send_msg` and `recv_msg` in `netio.h`, then run the pair provided:

```
./msg_server 5002
./msg_client localhost 5002 short "a longer message" x

short -> OK 5
a longer message -> OK 16
x -> OK 1
```

- **Q13.** The client sends all three messages before reading any reply. How does the server know where each one ends, even if all 34 bytes arrive in one `recv`? Why does `send_msg` use `htonl`?
- **Q14.** Write your own program `file_client.c` that sends a whole file as one message with `send_msg`, and check with `msg_server` that the reported size equals the file size (`stat -c %s file`). Try a file larger than 1 MB: what happens, and why?

Task 8 — Breaking things on purpose

Experiment	What to do	What to observe
Connection refused	Connect to a port where nothing listens.	`connect: Connection refused` — the host answered with a TCP RST.
Address already in use	Stop the server with Ctrl+C while a client is connected, comment out the `setsockopt(... SO_REUSEADDR ...)` line, rebuild and restart at once.	`bind: Address already in use` for about a minute: the old connection is in TIME_WAIT (`ss -tan | grep 5000`).
Peer closed	Kill the client with Ctrl+C while the server waits in `recv`.	`recv` returns 0; the server prints "client closed".
Broken pipe	Make a client that writes to a connection after the server has closed it.	The program is killed by SIGPIPE; with `signal(SIGPIPE, SIG_IGN)`, `send` fails with EPIPE instead.

- **Q15.** For each experiment, copy the exact error text and name the TCP mechanism behind it.
- **Q16.** Why does a server need SO_REUSEADDR during development? Is it safe to use in production?

The same programs in Python

Python's `socket` module is a thin layer over the same system calls; `sendall` is `send_all`. The archive contains `python/echo_server.py` and `python/echo_client.py`, which run unchanged on Linux and Windows. Compare them with the C versions line by line:

```
"""echo_server.py - iterative TCP echo server (Linux and Windows). Usage: python
echo_server.py PORT"""
import socket, sys

port = int(sys.argv[1])
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as ls:
    ls.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    ls.bind(("127.0.0.1", port))
    ls.listen(8)
    print(f"listening on 127.0.0.1:{port}", flush=True)
    while True:
        conn, (ip, cport) = ls.accept()          # a NEW socket for this client
        print(f"connection from {ip}:{cport}", flush=True)
        with conn:
            total = 0
            while data := conn.recv(65536):      # b"" means the client closed
                print(f"  recv returned {len(data)} bytes", flush=True)
                total += len(data)
                conn.sendall(data)               # sendall = send_all in C
            print(f"client closed after {total} bytes", flush=True)
```

If something does not work

Symptom	Likely cause and what to try
`bind: Address already in use`	Another program uses the port, or an old connection is in TIME_WAIT. Pick another port or wait; check `ss -tlnp`.
The client hangs after sending	The server reads with a fixed-size `recv_all` waiting for more bytes than were sent, or neither side ever closes.
Garbage after the echoed text	A buffer was printed with `%s` without a terminating `\0`. Print with `fwrite(buf, 1, n, stdout)` instead.
Works on localhost, not between machines	The server bound to 127.0.0.1, or a firewall blocks the port (`sudo ufw status`).
`getaddrinfo: Name or service not known`	Typo in the host name, or no DNS. Try an IP address.

PART D · WEEK 3

Laboratory 3 (Windows): Sockets I — a TCP client and server with Winsock

Two-hour laboratory sheet (2 × 50 minutes) accompanying Lecture 3

Windows implements the Berkeley sockets interface as **Winsock 2**. The concepts and most of the calls are the same as on Linux; the differences are few but each one breaks a program that ignores it. This sheet follows the Linux sheet task by task, with the Winsock versions of the programs. Both Windows programs were compiled without warnings with MinGW-w64.

By the end of the laboratory you should be able to

- build Winsock programs with MinGW-w64 or Visual Studio;
- list the differences between Winsock and POSIX sockets and handle each one;
- run a TCP client and server and observe their sockets with PowerShell;
- show, with measurements, that a byte stream has no message boundaries;
- read Winsock error codes;
- run the same Python programs on Windows and on Linux.

Before you start

- **A C compiler.** Either **MinGW-w64** (for example from MSYS2: `pacman -S mingw-w64-ucrt-x86_64-gcc`) or **Visual Studio Build Tools** with the "Desktop development with C++" workload; with the latter, use the *Developer PowerShell for VS*.
- **Building.** MinGW: `gcc echo_server_win.c -o echo_server.exe -lws2_32`. Visual Studio: `cl echo_server_win.c ws2_32.lib`. The library **ws2_32** must always be linked.
- **Files.** `netio_win.h`, `echo_client_win.c`, `echo_server_win.c`, and the `python/` folder.
- **Firewall.** The server binds to **127.0.0.1** only, so Windows Firewall does not interrupt you. Binding to all addresses (`INADDR_ANY`) makes Windows ask for permission the first time.
- **What to hand in.** Your sources and answers to **Q1–Q16**.

What is different in Winsock

	Linux (POSIX)	Windows (Winsock 2)
Initialisation	none	`WSAStartup(MAKEWORD(2,2), &wsa)` before any call; `WSACleanup()` at the end
Headers and library	`<sys/socket.h>`, `<netinet/in.h>`, `<arpa/inet.h>`, `<netdb.h>`	`<winsock2.h>`, `<ws2tcpip.h>`; link **ws2_32**
Socket type	`int` (a file descriptor)	`SOCKET` (an unsigned handle)
Failure value	`-1`	`INVALID_SOCKET` (socket, accept) or `SOCKET_ERROR` (others)
Closing	`close(fd)`	`closesocket(s)` — `close` does not work on sockets
Error code	`errno`, `perror`	`WSAGetLastError()`
Reading and writing	`read`/`write` or `recv`/`send`	only `recv`/`send`; lengths are `int`

	Linux (POSIX)	Windows (Winsock 2)
Common errors	ECONNREFUSED, EADDRINUSE, ECONNRESET	WSAECONNREFUSED (10061), WSAEADDRINUSE (10048), WSAECONNRESET (10054)
Broken pipe	SIGPIPE signal	no signal; `send` fails with WSAECONNRESET or WSAECONNABORTED

Plan

Step	Activity	Time
1	Warm-up: a listening socket seen from PowerShell	10 min
2	Build and run the echo client	15 min
3	Build and run the echo server	20 min
4	Sockets as Windows sees them	5 min
	Break	10 min
5	A stream is not a sequence of messages	15 min
6	Framing with lines	10 min
7	Framing with a length prefix	15 min
8	Breaking things on purpose	10 min

Task 1 — A listening socket seen from PowerShell

Windows has no nc, so start the Python server from the archive and look at it:

```
python python\echo_server.py 5000                                    # window A
Get-NetTCPConnection -LocalPort 5000                                  # window B
Test-NetConnection 127.0.0.1 -Port 5000
Get-NetTCPConnection   -LocalPort   5000   |   Format-Table   LocalAddress,   LocalPort,
RemoteAddress, RemotePort, State, OwningProcess
```

- **Q1.** Which states do you see before and after `Test-NetConnection`? Write the 4-tuple of the connection it made. Who chose the client's port?
- **Q2.** `Test-NetConnection` connects and closes at once. What does the server window print, and which TCP message told the server the client had closed?

Task 2 — The echo client

The helpers are the same as on Linux, with Winsock types:

```
/* netio_win.h - helpers for TCP byte streams on Windows (Winsock 2) - Computer Networks,
Lab 3 */
#ifndef NETIO_WIN_H
#define NETIO_WIN_H
#include <winsock2.h>
#include <ws2tcpip.h>
#include <stdint.h>

/* Write exactly len bytes. Returns 0 on success, -1 on error. */
static inline int send_all(SOCKET s, const void *buf, size_t len) {
    const char *p = buf;
    while (len > 0) {
```

```
        int n = send(s, p, (int)len, 0);
        if (n == SOCKET_ERROR) return -1;
        p += n; len -= (size_t)n;
    }
    return 0;
}

/* Read exactly len bytes. Returns 1 on success, 0 if the peer closed first, -1 on error.
*/
static inline int recv_all(SOCKET s, void *buf, size_t len) {
    char *p = buf;
    while (len > 0) {
        int n = recv(s, p, (int)len, 0);
        if (n == SOCKET_ERROR) return -1;
        if (n == 0) return 0;
        p += n; len -= (size_t)n;
    }
    return 1;
}

static inline int send_msg(SOCKET s, const void *buf, uint32_t len) {
    uint32_t netlen = htonl(len);
    if (send_all(s, &netlen, sizeof netlen) < 0) return -1;
    return send_all(s, buf, len);
}

static inline long recv_msg(SOCKET s, void *buf, uint32_t cap) {
    uint32_t netlen;
    int r = recv_all(s, &netlen, sizeof netlen);
    if (r <= 0) return r;
    uint32_t len = ntohl(netlen);
    if (len > cap) return -1;
    r = recv_all(s, buf, len);
    return r <= 0 ? -1 : (long)len;
}
#endif
```

```
/* echo_client_win.c - Winsock version of echo_client.c.
   Build:  gcc echo_client_win.c -o echo_client.exe -lws2_32     (MinGW-w64)
      or:  cl echo_client_win.c ws2_32.lib                        (Visual Studio)
   Usage:  echo_client.exe HOST PORT */
#include <stdio.h>
#include <string.h>
#include "netio_win.h"
#ifdef _MSC_VER
#pragma comment(lib, "ws2_32.lib")
#endif

int main(int argc, char *argv[]) {
    if (argc != 3) { fprintf(stderr, "usage: %s HOST PORT\n", argv[0]); return 1; }
    WSADATA wsa;
     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) { fprintf(stderr, "WSAStartup failed\n");
return 1; }

    struct addrinfo hints = {0}, *res, *ai;
    hints.ai_family = AF_UNSPEC; hints.ai_socktype = SOCK_STREAM;
    int err = getaddrinfo(argv[1], argv[2], &hints, &res);
    if (err) { fprintf(stderr, "getaddrinfo failed: %d\n", err); WSACleanup(); return 1; }

    SOCKET s = INVALID_SOCKET;
    for (ai = res; ai != NULL; ai = ai->ai_next) {
        s = socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol);
        if (s == INVALID_SOCKET) continue;
        if (connect(s, ai->ai_addr, (int)ai->ai_addrlen) == 0) break;
        closesocket(s); s = INVALID_SOCKET;
    }
```

```
    freeaddrinfo(res);
        if (s == INVALID_SOCKET) { fprintf(stderr, "connect failed: error %d\n",
WSAGetLastError()); WSACleanup(); return 1; }

    char line[1024], reply[1024];
    while (fgets(line, sizeof line, stdin) != NULL) {
        size_t len = strlen(line);
            if (send_all(s, line, len) < 0) { fprintf(stderr, "send failed: %d\n",
WSAGetLastError()); break; }
        int r = recv_all(s, reply, len);
         if (r <= 0) { fprintf(stderr, r == 0 ? "server closed the connection\n" : "recv
failed\n"); break; }
        fwrite(reply, 1, len, stdout);
    }
    closesocket(s);
    WSACleanup();
    return 0;
}
```

```
gcc echo_client_win.c -o echo_client.exe -lws2_32
.\echo_client.exe localhost 5000
```

- **Q3.** Find in the code every Winsock difference listed in the table above. Which one would make the program fail immediately if forgotten?
- **Q4.** Run the client while nothing listens on port 5000. Which error number does it print? Look it up (`net helpmsg` does not cover Winsock; search for “Windows Sockets error codes”). Why does the failure take about two seconds on Windows, but is immediate on Linux?

Task 3 — The echo server

```
/* echo_server_win.c - Winsock version of echo_server.c (iterative, one client at a time).
   Build:  gcc echo_server_win.c -o echo_server.exe -lws2_32
   Usage:  echo_server.exe PORT */
#include <stdio.h>
#include <stdlib.h>
#include "netio_win.h"
#ifdef _MSC_VER
#pragma comment(lib, "ws2_32.lib")
#endif

int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "usage: %s PORT\n", argv[0]); return 1; }
    setvbuf(stdout, NULL, _IONBF, 0);
    WSADATA wsa;
     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) { fprintf(stderr, "WSAStartup failed\n");
return 1; }

    SOCKET ls = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (ls == INVALID_SOCKET) { fprintf(stderr, "socket: %d\n", WSAGetLastError()); return
1; }

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
     /* Loopback only: avoids the Windows Firewall prompt. Use INADDR_ANY to accept other
machines. */
    addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    addr.sin_port = htons((u_short)atoi(argv[1]));
    if (bind(ls, (struct sockaddr *)&addr, sizeof addr) == SOCKET_ERROR) { fprintf(stderr,
"bind: %d\n", WSAGetLastError()); return 1; }
         if (listen(ls, 8) == SOCKET_ERROR) { fprintf(stderr, "listen: %d\n",
WSAGetLastError()); return 1; }
    printf("listening on 127.0.0.1:%s\n", argv[1]);

    for (;;) {
```

```
        struct sockaddr_in peer; int plen = sizeof peer;
        SOCKET cs = accept(ls, (struct sockaddr *)&peer, &plen);
         if (cs == INVALID_SOCKET) { fprintf(stderr, "accept: %d\n", WSAGetLastError());
continue; }
        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &peer.sin_addr, ip, sizeof ip);
        printf("connection from %s:%u\n", ip, ntohs(peer.sin_port));

        char buf[65536]; int n; long total = 0;
        while ((n = recv(cs, buf, sizeof buf, 0)) > 0) {
            printf("  recv returned %d bytes\n", n);
            total += n;
            if (send_all(cs, buf, (size_t)n) < 0) break;
        }
        printf("client closed after %ld bytes\n", total);
        closesocket(cs);
    }
}
```

```
gcc echo_server_win.c -o echo_server.exe -lws2_32
.\echo_server.exe 5000                    # window A
.\echo_client.exe 127.0.0.1 5000          # window B
.\echo_client.exe 127.0.0.1 5000          # window C, while B is still connected
```

- **Q5.** Match the server's calls with Figure 2 in Lecture 3. What does the server print for each client?
- **Q6.** Why does the client in window C get no echo until B quits? Is its TCP connection established meanwhile (`Get-NetTCPConnection -LocalPort 5000`)?
- **Q7.** Replace `htons((u_short)atoi(argv[1]))` with `(u_short)atoi(argv[1])`, rebuild and start the server on port 5000. On which port does it actually listen? Explain.

Task 4 — Sockets as Windows sees them

```
Get-NetTCPConnection -LocalPort 5000 | Select-Object State, RemotePort, OwningProcess
Get-Process -Id (Get-NetTCPConnection -LocalPort 5000 -State Listen).OwningProcess
netstat -ano | Select-String 5000
```

- **Q8.** With one client connected, how many sockets belong to the server process and in which states? Which program owns the other end?

Task 5 — A stream is not a sequence of messages

Send 300 000 bytes in **one** call from PowerShell and watch the server's `recv` sizes:

```
$c = New-Object System.Net.Sockets.TcpClient('127.0.0.1', 5000)
$s = $c.GetStream()
$data = [byte[]]::new(300000)
$s.Write($data, 0, $data.Length)          # one write of 300 000 bytes
$c.Client.Shutdown('Send')                # like shutdown(SHUT_WR)
$buf = [byte[]]::new(65536); while ($s.Read($buf, 0, $buf.Length) -gt 0) { }
$c.Close()
```

- **Q9.** One write — how many `recv` calls on the server? Repeat three times and with 10 and 3 000 000 bytes. Are the sizes the same each time?
- **Q10.** Change `sizeof buf` in the server's `recv` call to 7 and type `hello world` in the C client. What does the server print? Why does the client still get the whole echo?

Task 6 — Framing with lines

Port the line-handling loop of the Linux `upper_server.c` (shown in the Linux sheet, Task 6) to Winsock, using the table of differences. Test it with the PowerShell code of Task 5, writing the bytes

of `'abc' + [char]10 + 'def' + [char]10 + 'ghi'` instead of 300 000 zeros: use `$data = [Text.Encoding]::ASCII.GetBytes(...)`.

- **Q11.** Why is the last, incomplete line never answered?
- **Q12.** What should the server do with a line longer than its buffer?

Task 7 — Framing with a length prefix

`netio_win.h` also contains `send_msg` and `recv_msg`. Write `msg_server_win.c`, which replies `OK <n>` to every message of *n* bytes (the Linux `msg_server.c` is the model), and a client that sends several messages before reading the replies.

- **Q13.** How does the server find the end of each message even when several arrive in one `recv`? Why `htonl`?
- **Q14.** Send a file as one message and compare the reported length with `(Get-Item file).Length`.

Task 8 — Breaking things on purpose

Experiment	What to do	What to observe on Windows
Connection refused	Connect to a port where nothing listens.	Error 10061 (WSAECONNREFUSED), after about 2 s of retries.
Address already in use	Start two servers on the same port.	`bind: 10048` (WSAEADDRINUSE).
Peer closed	Close the client window while the server waits in `recv`.	`recv` returns 0 (normal close) or fails with 10054 (WSAECONNRESET) if the process was killed.
Writing to a closed connection	Send after the server has closed.	No signal as on Linux; `send` fails with 10053 or 10054.

- **Q15.** Copy each error number and explain the TCP mechanism behind it.
- **Q16.** Windows usually lets a restarted server bind its port again at once, where Linux needs SO_REUSEADDR. On Windows, SO_REUSEADDR has a different and dangerous meaning: it lets a second program bind to a port already in use. Find the Winsock option SO_EXCLUSIVEADDRUSE in the documentation and explain what it prevents.

The same programs in Python

The files in `python/` run unchanged on Windows: `python python\echo_server.py 5000` and `python python\echo_client.py 127.0.0.1 5000`. Python hides `WSAStartup` and the type differences; the stream behaviour — and the need for `recv_all` — is exactly the same.

```
"""echo_client.py - send lines from standard input, print the echo. Usage: python
echo_client.py HOST PORT"""
import socket, sys

def recv_all(sock, n):
    """Read exactly n bytes, or raise if the peer closes first."""
    chunks = []
    while n > 0:
        data = sock.recv(n)
        if not data:
            raise ConnectionError("server closed the connection")
        chunks.append(data)
```

```
        n -= len(data)
    return b"".join(chunks)

with socket.create_connection((sys.argv[1], int(sys.argv[2]))) as s:
    for line in sys.stdin:
        data = line.encode()
        s.sendall(data)
        print(recv_all(s, len(data)).decode(), end="")
```

If something does not work

Symptom	Likely cause and what to try
`undefined reference to __imp_socket` (MinGW)	The Winsock library was not linked: add `-lws2_32` at the end of the gcc command.
Every call fails with error 10093	WSANOTINITIALISED: `WSAStartup` was not called first.
The firewall asks for permission	The server binds to all addresses. For the lab, bind to 127.0.0.1 (as the provided code does).
`gcc` is not recognised	Open the MSYS2 UCRT64 terminal, or add its `bin` folder to PATH.
Python: `ConnectionRefusedError: [WinError 10061]`	Nothing listens on that port; start the server first.

Next week

Lecture 4 continues with the Web: cookies, caching, HTTP/2, HTTP/3 and content distribution networks. Laboratory 4 moves to UDP sockets and byte order.